

Muestreo

Diego Da Rosa

El Muestreo es la parte de la estadística que selecciona muestras de la población objetivo, observa las características de las unidades de muestreo seleccionadas y luego usar esas observaciones para hacer inferencias sobre toda la población objetivo.

En nuestro caso tenemos una población objetivo que son todos los clientes actuales de un banco, nuestra población objetivo tiene un tamaño de $N = 45,211$ observaciones. En nuestro dataset tenemos datos sobre clientes de un banco (edad, tipo de trabajo, saldo, si aceptaron un préstamo).

Nuestro Marco Muestral (Sampling Frame) es la base de datos bank.csv.

La Unidad de Muestreo es el cliente individual.

```
library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.4.3
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tibble' was built under R version 4.4.3
```

```
Warning: package 'tidyr' was built under R version 4.4.3
```

```
Warning: package 'readr' was built under R version 4.4.3
```

```
Warning: package 'purrr' was built under R version 4.4.3
```

```
Warning: package 'dplyr' was built under R version 4.4.3
```

```
Warning: package 'stringr' was built under R version 4.4.3
```

Warning: package 'forcats' was built under R version 4.4.3

Warning: package 'lubridate' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    4.0.0      v tibble     3.2.1
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
# Carga de datos (puedes descargarlo o leerlo directamente si tienes el link)
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank.zip"
temp <- tempfile()
download.file(url, temp)
bank_data <- read.table(unz(temp, "bank.csv"), sep = ";", header = TRUE)
unlink(temp)
```

```
# Ver la estructura
```

```
str(bank_data)
```

```
'data.frame':  4521 obs. of  17 variables:
 $ age      : int  30 33 35 30 59 35 36 39 41 43 ...
 $ job      : chr  "unemployed" "services" "management" "management" ...
 $ marital  : chr  "married" "married" "single" "married" ...
 $ education: chr  "primary" "secondary" "tertiary" "tertiary" ...
 $ default  : chr  "no" "no" "no" "no" ...
 $ balance  : int  1787 4789 1350 1476 0 747 307 147 221 -88 ...
 $ housing  : chr  "no" "yes" "yes" "yes" ...
 $ loan     : chr  "no" "yes" "no" "yes" ...
 $ contact  : chr  "cellular" "cellular" "cellular" "unknown" ...
 $ day      : int  19 11 16 3 5 23 14 6 14 17 ...
 $ month    : chr  "oct" "may" "apr" "jun" ...
 $ duration : int  79 220 185 199 226 141 341 151 57 313 ...
 $ campaign : int  1 1 1 4 1 2 1 2 2 1 ...
 $ pdays   : int  -1 339 330 -1 -1 176 330 -1 -1 147 ...
 $ previous : int  0 4 1 0 0 3 2 0 0 2 ...
```

```
$ poutcome : chr  "unknown" "failure" "failure" "unknown" ...
$ y         : chr  "no" "no" "no" "no" ...
```

```
#bank_data
```

```
sum(is.na(bank_data))
```

```
[1] 0
```

```
colnames(bank_data)
```

```
[1] "age"      "job"      "marital"  "education" "default"  "balance"
[7] "housing"  "loan"     "contact"  "day"       "month"    "duration"
[13] "campaign" "pdays"   "previous" "poutcome" "y"
```

En muestreo lo que hacemos es obtener una muestra para realizar una estimación del promedio de una variable de interés, en nuestro caso nuestra variable de interés es “balance” que representa el dinero que tiene un usuario del banco y nos interesa estimar el promedio de esta variable (que en realidad el promedio es estimar el total y dividir esa estimación entre el tamaño de la población si es conocido o entre el tamaño de la muestra) que es una variable continua con alta asimetría y valores atípicos, lo cual nos influirá al querer obtener una estimación del promedio de esta variable.

Luego contamos con variables auxiliares, que son todas las demás variables del set de datos, como job, education, marital, age, etc que puede o no tener una correlación con nuestra variables de interés que es balance, cuanto más correlación haya entre alguna de estas variables auxiliares y balance, mejores serán las estimaciones. Estas variables también nos servirán para diversos métodos que veremos más adelante como Estratificación, Raking, Post-estratificación.

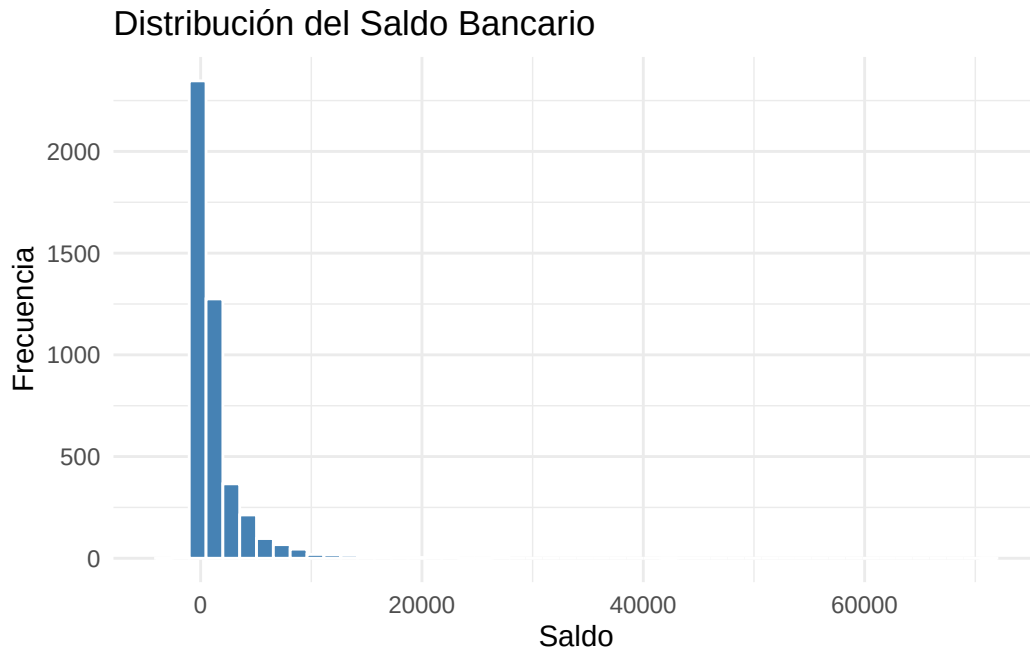
Los datos provienen de una base de datos institucional. Dado que los recursos para realizar encuestas a profundidad son limitados, se requiere un diseño muestral que capture la variabilidad del balance sin sesgar los resultados hacia grupos sobre-representados (como clientes con empleos administrativos).

Si empezamos explorando la distribución de nuestra variable de interés balance

```
# Resumen estadístico
summary(bank_data$balance)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-3313	69	444	1423	1480	71188

```
# Visualización de la distribución
library(ggplot2)
ggplot(bank_data, aes(x = balance)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribución del Saldo Bancario", x = "Saldo", y = "Frecuencia")
```



Vemos que tenemos asimetría extrema con un sesgo a la derecha.

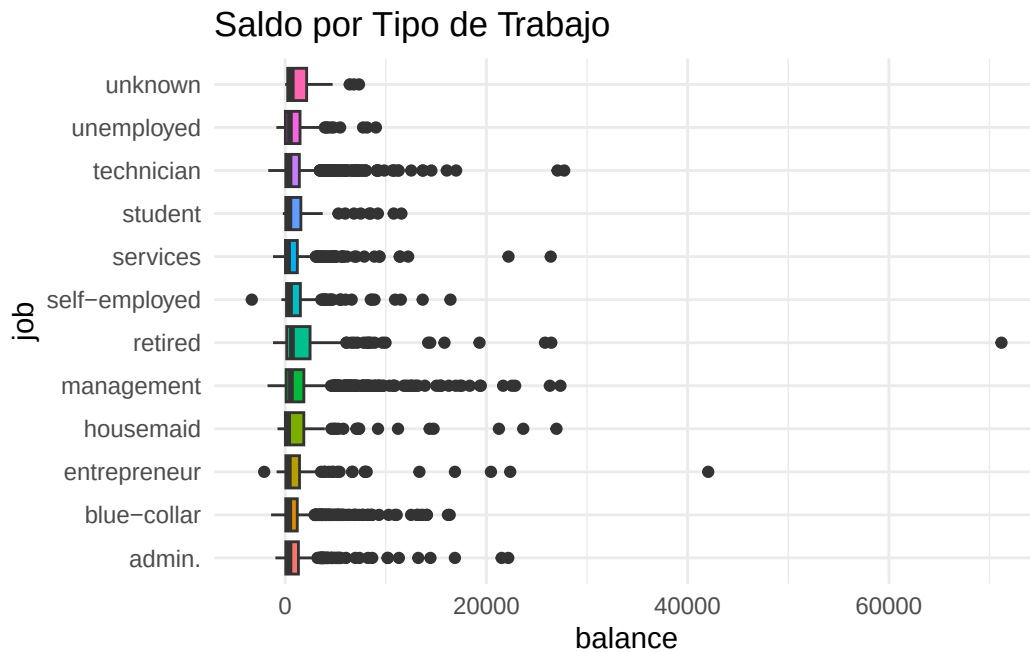
La gráfica muestra una concentración masiva de clientes en saldos bajos, cercanos a 0, y una cola muy larga hacia la derecha. Esta asimetría nos causara problemas mas adelante los cuales resolveremos.

Si hacemos un Muestreo Aleatorio Simple (MAS), tienes una probabilidad altísima de no seleccionar a ninguno de los clientes con saldos altos (los que están en la cola derecha). Esto causaría una subestimación sistemática del saldo promedio del banco.

Ahora en cuanto a las distintas profesiones

```
ggplot(bank_data, aes(x = job, y = balance, fill = job)) +
  geom_boxplot() +
  coord_flip() + # Para leer mejor los nombres de los trabajos
  theme_minimal() +
```

```
theme(legend.position = "none") +
labs(title = "Saldo por Tipo de Trabajo")
```



Observamos los valores atípicos que pueden generar problemas al usar esta covariable para explicar a nuestra variable de interés balance, además vemos que hay grupos como retired (jubilados) o management que tienen cajas más anchas y bigotes más largos (más variabilidad), y hay otros grupos más uniformes como services o blue-collar. Además la variabilidad cambia según el trabajo, por lo que sería bueno que el diseño muestral a usar considere el empleo como variable de control, porque uno que no lo haga será ineficiente.

Para obtener la media real (de nuestra población) usamos el siguiente código y obtenemos el valor de 1422.658

```
# 1. Real
media_real <- mean(bank_data$balance)
media_real
```

```
[1] 1422.658
```

Muestreo Aleatorio Simple sin Reemplazo

Todos los clientes de tu base de datos tienen exactamente la misma probabilidad de ser seleccionados, la cual es $\pi_i = \frac{n}{N}$, es decir, la probabilidad de seleccionar al individuo i es el tamaño

de la muestra dividido el tamaño de la población, por ejemplo si nuestra población tuviera 1000 datos y sacamos una muestra de 100 personas, la probabilidad de que cualquier persona de la muestra sea seleccionado es $100/1000 = 0.1$. Este diseño cumple el supuesto de Independencia, el cual dice que la selección de un cliente con saldo alto no afecta la probabilidad de seleccionar a otro con saldo bajo. Este diseño considera que el tamaño de la muestra es fijo, que no varía y nosotros podemos obtener una muestra de un tamaño que nos interese. En este MAS asumimos que no hay reemplazo, cuando sale un elemento de la muestra, entonces no puede volver a salir de vuelta.

Realizamos una estimación de la media poblacional usando una muestra de tamaño 500, obteniendo el valor de 1458.8

```
# 2. MAS (Simple)

# Selección de una muestra aleatoria simple
n <- 500
muestra_mas <- bank_data[sample(nrow(bank_data), n), ]

# Definición del diseño en el paquete survey
library(survey)
```

Warning: package 'survey' was built under R version 4.4.3

Cargando paquete requerido: grid

Cargando paquete requerido: Matrix

Adjuntando el paquete: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Cargando paquete requerido: survival

Adjuntando el paquete: 'survey'

The following object is masked from 'package:graphics':

dotchart

```
diseño_mas <- svydesign(id = ~1, data = muestra_mas)
```

Warning in svydesign.default(id = ~1, data = muestra_mas): No weights or probabilities supplied, assuming equal probability

```
# Estimación de la media del balance
res_mas <- svymean(~balance, diseño_mas)
ic_mas <- confint(res_mas)
res_mas
```

```
      mean      SE
balance 1307.9 109.19
```

```
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media MAS sin Reemplazo:", round(res_mas, 2)))
```

```
[1] "Media MAS sin Reemplazo: 1307.94"
```

Ahora el problema que tenemos con este diseño es que como vimos en el histograma de saldos, el MAS será ineficiente porque hay mucha variabilidad y una asimetría extrema, esto causa que la muestra aleatoria simple no incluya suficientes individuos de la “cola derecha”, los usuarios de saldos altos, lo que resultaría en una estimación sesgada y errónea del promedio de la riqueza de los usuarios del banco. En nuestro caso vemos que la estimación del promedio hecha con la muestra está considerablemente alejada de la media real.

Diseño Bernoulli

Este Diseño asume que cada unidad de la población del banco tiene una probabilidad de inclusión independiente e idéntica (π). A diferencia del Muestreo Aleatorio Simple (MAS), donde el tamaño de la muestra (n) es fijo, en el diseño Bernoulli el tamaño de la muestra es aleatorio, varía según la extracción que hagamos, no siempre es el mismo. Se puede probar

matemáticamente que el número final de clientes seleccionados sigue una distribución Binomial $B(N, \pi)$ y en promedio tendremos $N \times \pi$ clientes, pero el número exacto variará cada vez que ejecutemos el proceso.

Este Diseño sigue cumpliendo la Independencia, donde la selección de un cliente no afecta en absoluto la probabilidad de seleccionar a otro.

```
library(survey)

# 1. Definir la probabilidad de inclusión (pi)
pi <- 0.05
N <- nrow(bank_data)

# 2. Ejecutar el muestreo Bernoulli
# Generamos una variable indicadora (0 o 1) para cada registro
set.seed(123)
seleccion <- rbinom(N, 1, pi)
muestra_bernoulli <- bank_data[seleccion == 1, ]

# 3. Definir el diseño en el paquete survey
# En el diseño Bernoulli, la probabilidad de inclusión es constante
diseno_bernoulli <- svydesign(
  id = ~1,
  prob = ~rep(pi, nrow(muestra_bernoulli)),
  data = muestra_bernoulli
)

# 4. Estimar la media de la variable 'balance'
estimacion_media <- svymean(~balance, diseno_bernoulli)

# 5. Mostrar resultados e Intervalo de Confianza
print(estimacion_media)
```

```
      mean      SE
balance 1328.1 158.66
```

```
confint(estimacion_media)
```

```
      2.5 %  97.5 %
balance 1017.134 1639.07
```



```
# Ver el tamaño de muestra obtenido
cat("Tamaño de muestra esperado:", N * pi, "\n")
```

Tamaño de muestra esperado: 226.05

```
cat("Tamaño de muestra obtenido:", nrow(muestra_bernoulli), "\n")
```

Tamaño de muestra obtenido: 216

```
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media Bernoulli:", round(estimacion_media, 2)))
```

```
[1] "Media Bernoulli: 1328.1"
```

Vemos que la estimacion esta bastante alejada de el valor poblacional de la media, esto se debe a que el diseño Bernoulli tambien se ve afectado por la asimetría observada en el histograma de saldos, donde la variable balance tiene una cola derecha muy larga. En un diseño Bernoulli con una π baja, hay un alto riesgo de que el tamaño de muestra efectivo sea pequeño en los sectores de saldos altos, lo que sube el error estándar. Además la varianza en el diseño Bernoulli es mayor por la incertidumbre adicional sobre cuántos sujetos terminarán en la muestra.

Si queremos hacer una tabla comparativa:

```
# Comparación rápida
data.frame(Metodo = c("Real", "Simple", "Bernoulli"),
           Media = c(media_real, res_mas[1], estimacion_media))
```

	Metodo	Media
1	Real	1422.658
2	Simple	1307.944
3	Bernoulli	1328.102

Ahora debemos tener cuidado con las interpretaciones ya que estas estimaciones se hicieron con una ÚNICA muestra, generalmente los estimadores, que son funciones de los datos muestrales se evalúan en múltiples muestras, para las cuales los estimadores tomaran valores distintos, obteniendo así una distribución de probabilidad para estos estimadores. Probaremos esto usando 1000 muestras para evaluar el rendimiento del MAS contra el Bernoulli.

```

set.seed(123)
iteraciones <- 1000
N <- nrow(bank_data)
pi_prob <- 0.05
n_fijo <- round(N * pi_prob)

# Crear contenedor para los resultados
simulacion <- data.frame(
  MAS = numeric(iteraciones),
  Bernoulli = numeric(iteraciones)
)

for(i in 1:iteraciones) {
  # 1. Simulación MAS (n fijo)
  muestra_mas <- bank_data[sample(N, n_fijo), ]
  simulacion$MAS[i] <- mean(muestra_mas$balance)

  # 2. Simulación Bernoulli (n aleatorio)
  # Seleccionamos con probabilidad pi_prob cada fila
  seleccion_bern <- rbinom(N, 1, pi_prob)
  muestra_bern <- bank_data[seleccion_bern == 1, ]

  # Estimador de Horvitz-Thompson para la media en Bernoulli:
  # (Suma de valores observados) / (N * pi)
  simulacion$Bernoulli[i] <- sum(muestra_bern$balance) / (N * pi_prob)
}

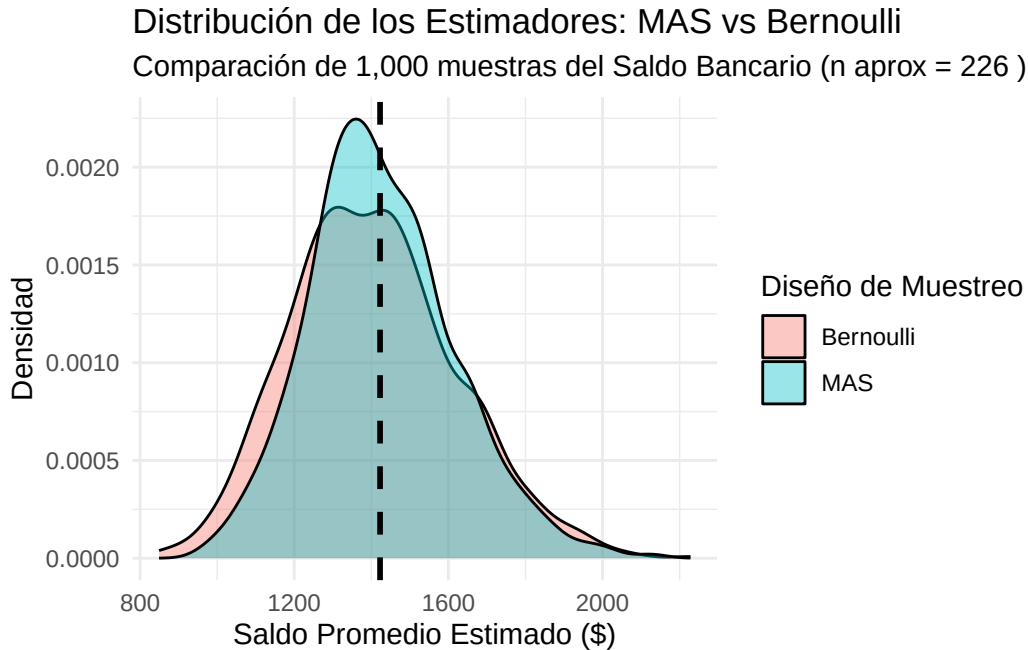
# 3. Preparar datos para ggplot (usando tidyr en lugar de reshape2 para evitar avisos)
library(tidyr)
sim_long <- pivot_longer(simulacion, cols = everything(), names_to = "Diseno", values_to = "Media")

# 4. Graficar comparativa corregida
library(ggplot2)
ggplot(sim_long, aes(x = Media, fill = Diseno)) +
  geom_density(alpha = 0.4) +
  # CORRECCIÓN: xintercept en lugar de x_intercept
  geom_vline(xintercept = mean(bank_data$balance), linetype = "dashed", color = "black", size = 1) +
  labs(title = "Distribución de los Estimadores: MAS vs Bernoulli",
        subtitle = paste("Comparación de 1,000 muestras del Saldo Bancario (n aprox =", n_fijo, ")",
                          "x = \"Saldo Promedio Estimado ($)\"",
                          "y = \"Densidad\",
                          fill = \"Diseño de Muestreo\") +

```

```
theme_minimal()
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



Vemos en el gráfico de Densidad de los Estimadores, que si bien ambos diseños son insesgados, es decir, en promedio tienden a estimar bien el valor promedio real, y esto se observa en que las curvas de los estimadores se centran correctamente en la media real de 1422.658, la curva azul del MAS es notablemente más estrecha y alta. Esto demuestra que el MAS es un diseño más preciso para este banco. La mayor dispersión de la curva roja de Bernoulli se debe a que, en ese diseño, el tamaño de la muestra es aleatorio; esa incertidumbre sobre cuántos clientes terminan siendo encuestados añade variabilidad que sube el error. Aunque seguimos teniendo los problemas mencionados de asimetría, alta variabilidad, y no consideración de usuarios bancarios con alto ingreso.

Al tener los estimadores distribucino de probabilidad, podemos calcular distintos estadisticos como

El sesgo es la comparacion entre la media obtenida con los estimadores usando las multiples muestra extraidas y la media poblacional, el sesgo mide la exactitud de nuestras estimaciones. Un sesgo cercano a cero indica que tu diseño no tiene preferencias ni errores sistemáticos, no está inflado ni subestimado.

La Varianza mide la precisión o estabilidad, vimos que el saldo bancario es muy volátil (con clientes de \$0 y otros de \$60,000), una varianza alta hace que con cada muestra extraída el resultado pueda cambiar mucho.

El MSE (Error Cuadrático Medio) indica el error total, es la suma del sesgo elevado al cuadrado y de la varianza. Esta métrica sirve para decidir qué diseño es mejor, a menor MSE, más eficiente es el muestreo.

El RMSE da el error promedio en las mismas unidades que el saldo (que esta en dolares), si el RMSE es de \$200, significa que, en promedio, las estimaciones se desvían \$200 del valor real.

El CV (Coeficiente de Variación) da la incertidumbre como un porcentaje. Un CV menor al 5% o 10% indica que el error es pequeño en relación con la magnitud de los saldos que estamos midiendo.

Si hacemos la tabla comparativa del MAS y el Bernoulli

```
library(tidyverse)

# Asumiendo que ya tienes el dataframe 'resultados' de la simulación anterior
# con las columnas 'metodo' y 'media'

# Calculamos la media poblacional real como parámetro de referencia
media_real <- mean(bank_data$balance)

# Transformamos los resultados de la simulación para el análisis
metricas <- sim_long %>%
  group_by(Disenos) %>%
  summarise(
    Media_Estimadores = mean(Media),
    Sesgo = mean(Media) - media_real,
    Varianza = var(Media),
    MSE = mean((Media - media_real)^2),
    RMSE = sqrt(MSE),
    # Coeficiente de Variación del estimador (mide precisión relativa)
    CV = (sd(Media) / mean(Media)) * 100
  )

print(metricas)
```

```
# A tibble: 2 x 7
  Disenos Media_Estimadores Sesgo Varianza MSE RMSE CV
  <chr>          <dbl>   <dbl>   <dbl> <dbl> <dbl> <dbl>
```

1 Bernoulli	1405.	-17.2	45807.	46058.	215.	15.2
2 MAS	1426.	3.77	35167.	35146.	187.	13.1

Podemos decir en general que el MAS es mejor que el Bernoulli aunque a gran escala tenemos el problema de asimetría y selección sesgada dada la poca presencia de clientes con mucho dinero en el banco.

Muestreo Aleatorio Simple con Reemplazo

El Muestreo Aleatorio Simple con Reemplazo (SIR) es una variante del MAS donde cada vez que seleccionas a un cliente de la población para ponerlo en la muestra, este puede aparecer dos o más veces en la muestra final. La varianza del estimador bajo posibilidad de reemplazo suele ser mayor que si no hubiera posibilidad de reemplazo, porque al permitir repeticiones, la muestra es menos informativa, tenemos menos individuos únicos para representar a la población.

```
# 1. Definir parámetros
n <- 500
N <- nrow(bank_data)

# 2. Extraer la muestra SIR (con reemplazo)
set.seed(123)
indices_sir <- sample(1:N, size = n, replace = TRUE)
muestra_sir <- bank_data[indices_sir, ]

# 3. Verificar si hay duplicados
n_unicos <- length(unique(indices_sir))
cat("Individuos únicos seleccionados:", n_unicos, "de", n, "\n")
```

Individuos únicos seleccionados: 476 de 500

```
# 4. Definir el diseño en el paquete survey
# Para SIR, no se incluye el argumento 'fpc'
diseno_sir <- svydesign(
  id = ~1,
  weights = ~rep(N/n, n),
  data = muestra_sir
)

# 5. Estimar la media del balance
res_sir <- svymean(~balance, diseno_sir)
print(res_sir)
```

	mean	SE
balance	1402.6	118.59

```
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media MAS con Reemplazo:", round(res_sir, 2)))
```

```
[1] "Media MAS con Reemplazo: 1402.6"
```

En media, tanto reemplazo como no reemplazo son insesgados (muy cerca del valor real). En Varianza y RMSE, con reemplazo es mayor debido a la reduccion de la diversidad de la muestra. En CV (Coeficiente de Variación), con reemplazo tendra una precisión menor.

Hasta ahora la lógica detrás de todas las estimaciones se basaba en expandir la muestra y estabamos usando el estimador Horvitz-Thompson (HT). Con la formula

$$\hat{Y}_{HT} = \sum_{i \in s} \frac{y_i}{\pi_i} = \sum_{i \in s} y_i \cdot w_i$$

Lo que hace es para todos los individuos de la muestra, dividirlos entre su probabilidad de seleccion. Este estimador siempre insesgado, pero se usa solo para diseños sin reemplazo. Para el caso de diseños con reemplazo, usamos el estimador Hansen-Hurwitz (pwr).

$$\hat{Y}_{pwr} = \frac{1}{n} \sum \frac{y_i}{p_i}$$

Estimador pwr

```
library(tidyverse)

# 1. Definir probabilidades de selección (p_i)
# Deben sumar 1 en toda la población
bank_data <- bank_data %>%
  mutate(prioridad = case_when(
    education == "tertiary" ~ 3,
    education == "secondary" ~ 2,
    TRUE ~ 1
  )) %>%
  mutate(p_i = prioridad / sum(prioridad))
```

```
# 2. Realizar el muestreo CON reemplazo (n = 500)
n_muestras <- 500
set.seed(789)
indices_pwr <- sample(1:nrow(bank_data), size = n_muestras, replace = TRUE, prob = bank_data$balance)
muestra_pwr <- bank_data[indices_pwr, ]

# 3. Calcular el Estimador pwr para el TOTAL
# Aplicamos la fórmula: (1/n) * suma(y_i / p_i)
total_pwr <- (1 / n_muestras) * sum(muestra_pwr$balance / muestra_pwr$p_i)

# 4. Calcular la MEDIA estimada
media_pwr <- total_pwr / nrow(bank_data)

# Comparación
media_real <- mean(bank_data$balance)
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media Estimador pwr:", round(media_pwr, 2)))
```

```
[1] "Media Estimador pwr: 1218.67"
```

Si comparamos ambos usando un diseño simple (sin reemplazo para el HT y con reemplazo para el pwr) vemos que las estimaciones de pwr se aleja mas de la media real que es 1422.658, mientras que HT se acerca bastante

Esto se debe a que el diseño PWR (Hansen-Hurwitz) usa el saldo como medida de probabilidad. Este estimador es muy sensible a la relación entre el saldo y su peso. Si la mayoría de los clientes tienen saldos bajos (como se ve en el histograma), el PWR tiende a estabilizarse en esos valores. Al asignar probabilidades altas a los saldos grandes, el diseño selecciona repetidamente a los mismos clientes extremos (recuerda que es con reemplazo), la varianza del estimador se dispara

Para el HT, la media vale 1735.6, es más alta porque, en un muestreo aleatorio simple, si atrapamos por azar a uno de los pocos clientes con \$60,000, ese valor empuja el promedio hacia arriba sin ninguna ponderación que lo suavice. Aunque conceptualmente tiene mas sentido usar un diseño sin reemplazo porque repetir los ingresos de los usuarios del banco tiende a ser informacion redundante.

```

library(survey)
library(dplyr)
library(tidyr)

# 1. Limpieza rigurosa: Solo saldos positivos y sin NAs
bank_clean <- bank_data %>%
  filter(balance > 0) %>%
  drop_na(balance)

N_pob <- nrow(bank_clean)
n_muest <- 500

# 2. Definir Probabilidades de Selección (p_i) por extracción
# Estas deben sumar exactamente 1
p_pob <- bank_clean$balance / sum(bank_clean$balance)

# 3. Muestreo con Reemplazo (PPS)
set.seed(123)
indices <- sample(1:N_pob, size = n_muest, replace = TRUE, prob = p_pob)
muestra_final <- bank_clean[indices, ]

# Extraemos las probabilidades específicas de los elementos seleccionados
p_muest <- p_pob[indices]

# -----
# A. DISEÑO PWR (Hansen-Hurwitz)
# -----
# Usamos 'probs' para las probabilidades de selección en cada extracción
diseno_pwr <- svydesign(
  id = ~1,
  probs = ~p_muest,
  data = muestra_final
)

# -----
# B. DISEÑO HT (Muestreo Aleatorio Simple como base)
# -----
# Para comparar, extraemos una muestra MAS de la misma población limpia
set.seed(123)
muestra_mas <- bank_clean[sample(N_pob, n_muest, replace = FALSE), ]
diseno_ht <- svydesign(
  id = ~1,

```



```

    fpc = rep(N_pob, n_muest),
    data = muestra_mas
)

# 4. Cálculo de Métricas Comparativas
res_pwr <- svymean(~balance, diseno_pwr)
res_ht <- svymean(~balance, diseno_ht)

# print(res_pwr)
# print(res_ht)

print(paste("Media Estimador HT:", round(res_pwr, 2)))

```

```
[1] "Media Estimador HT: 1599.81"
```

```
print(paste("Media Estimador pwr:", round(res_ht, 2)))
```

```
[1] "Media Estimador pwr: 1702.63"
```

```
cat("\n")
```

```

# Extraer las varianzas directamente de los objetos de estimación
var_pwr <- vcov(res_pwr)[1,1]
var_ht <- vcov(res_ht)[1,1]

# Calcular la Eficiencia Relativa
eficiencia_relativa <- var_ht / var_pwr

cat("Varianza HT (MAS):", var_ht, "\n")

```

```
Varianza HT (MAS): 21545.08
```

```
cat("Varianza PWR (PPS):", var_pwr, "\n")
```

```
Varianza PWR (PPS): 33192.43
```

```
cat("Eficiencia Relativa (HT/PWR):", eficiencia_relativa, "\n")
```

Eficiencia Relativa (HT/PWR): 0.6490962

Por los factores que mencionamos relacionados a que la varianza de diseños con reemplazo es mas alta que diseños sin reemplazo, vemos que en este caso se cumple, siendo la varianza del pwr casi 6 veces mas grande que la varianza del HT.

Diseño Estratificado

El Muestreo Estratificado es una técnica de diseño donde la población se divide en grupos mutuamente excluyentes llamados estratos, y luego se selecciona una muestra independiente dentro de cada uno de ellos. Nosotros decidimos la estructura de la muestra para asegurar que todos los sectores importantes del banco estén representados.

Como vimos en el boxplot, los estratos los podemos hacer con las categorías de empleo del banco, esto debido a:

- Homogeneidad Interna: En el boxplot, los saldos de los “estudiantes” se parecen mucho entre sí, pero son muy diferentes a los de los “jubilados”. Al estratificar, el error estándar solo depende de la variación dentro de cada grupo, eliminando la gran diferencia que hay entre grupos.
- Protección contra “Malas Muestras”: En el MAS, existe el riesgo de que tu muestra no incluya a nadie de una categoría pequeña pero importante (como los desempleados). En el estratificado, garantizas una cuota mínima para cada profesión.
- Mayor Precisión (Menor Varianza): Matemáticamente, si los estratos están bien elegidos (es decir, si la profesión realmente influye en el saldo), la varianza del estimador estratificado siempre será menor o igual a la del muestreo aleatorio simple.

En cada estrato podemos aplicar el diseño que querramos, simple, bernoulli, etc.

```
library(tidyverse)
library(sampling)
```

Warning: package 'sampling' was built under R version 4.4.3

Adjuntando el paquete: 'sampling'

The following objects are masked from 'package:survival':

cluster, strata

```
# 1. Preparar los datos (ordenar por estrato es buena práctica)
bank_data <- bank_data %>% arrange(job)
N <- nrow(bank_data)
n_total <- 500

# 2. Calcular tamaños de estrato en la población (Nh)
estratos_info <- bank_data %>%
  group_by(job) %>%
  summarise(Nh = n(), Sh = sd(balance)) %>%
  mutate(wh = Nh / N) # Pesos poblacionales

# 3. Afijación Proporcional (nh)
estratos_info <- estratos_info %>%
  mutate(n_prop = round(n_total * wh))

# 4. Extraer la muestra estratificada (Proporcional)
set.seed(123)
muestra_estrat <- strata(bank_data,
  stratanames = "job",
  size = estratos_info$n_prop,
  method = "srswor") # Sin reemplazo dentro del estrato

# 5. Recuperar los datos de la muestra
datos_muestra_estrat <- getdata(bank_data, muestra_estrat)
```

```
# Cálculo manual del estimador estratificado
estimacion_estrat <- datos_muestra_estrat %>%
  group_by(job) %>%
  summarise(media_h = mean(balance)) %>%
  left_join(estratos_info, by = "job") %>%
  summarise(media_global_st = sum(media_h * wh))

media_real <- mean(bank_data$balance)
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

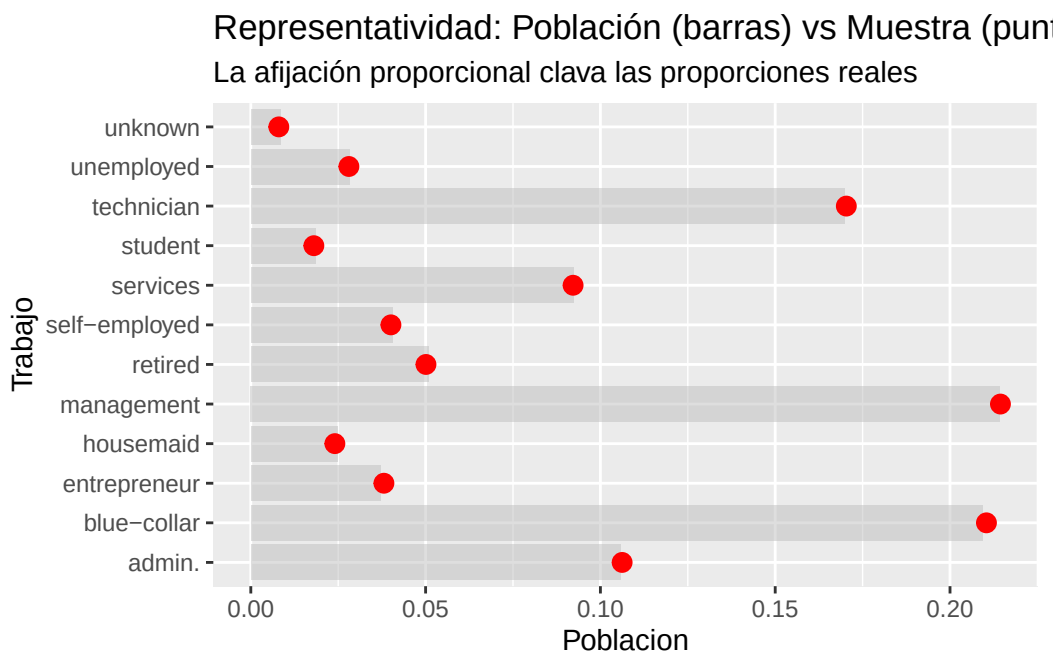
```
print(paste("Media Estratificada:", round(estimacion_estrat$media_global_st, 2)))
```

```
[1] "Media Estratificada: 1658.96"
```

Obtenemos una estimacion bastante buena.

```
# Comparar proporciones
comp_proporciones <- data.frame(
  Trabajo = estratos_info$job,
  Poblacion = estratos_info$wh,
  Muestra = as.vector(table(datos_muestra_estrat$job) / nrow(datos_muestra_estrat))
)

ggplot(comp_proporciones, aes(x = Trabajo)) +
  geom_bar(aes(y = Poblacion), stat = "identity", fill = "gray", alpha = 0.5) +
  geom_point(aes(y = Muestra), color = "red", size = 3) +
  coord_flip() +
  labs(title = "Representatividad: Población (barras) vs Muestra (puntos)",
       subtitle = "La afijación proporcional clava las proporciones reales")
```



Este gráfico compara el peso de cada profesión en la población total (barras grises) contra su presencia en la muestra (puntos rojos). Los puntos rojos caen aproximadamente al final de cada barra gris, se logra una Afijación Proporcional perfecta.

En un muestreo aleatorio simple, por puro azar, podrías haberte quedado sin “estudiantes” o con demasiados “jubilados”. Aquí forzamos que la muestra sea representativa de la población del banco. Al asegurar que cada grupo (estrato) esté representado en su justa medida, quitamos el error derivado de una muestra sesgada hacia profesiones con saldos muy altos o muy bajos.

Diseño Sistemático

En lugar de elegir a cada individuo al azar, seleccionamos a los elementos de la población a intervalos regulares de una lista ordenada. En nuestro caso

Si tenemos a todos los clientes ordenados en una lista, para aplicar el Diseño Sistemático hacemos un proceso de tres pasos:

- 1) Calcular el salto (Intervalo k): Si tienes $N = 45,211$ clientes y quieres una muestra de $n = 500$, tu intervalo será $k = N/n \approx 90$.
- 2) Arranque aleatorio (A): Eliges un número al azar entre 1 y 90 (por ejemplo, el 12).
- 3) Selección sistemática: El primer seleccionado es el cliente 12, el segundo es el $12 + 90$, el tercero el $12 + 180$, y así sucesivamente hasta recorrer toda la lista.

```
# 1. Parámetros
N <- nrow(bank_data)
n <- 500
k <- floor(N / n)

# 2. Arranque aleatorio
set.seed(123)
arranque <- sample(1:k, 1)

# 3. Selección de índices
indices_sist <- seq(from = arranque, to = N, by = k)

# 4. Crear la muestra
muestra_sistemica <- bank_data[indices_sist, ]

# Verificación
nrow(muestra_sistemica)
```

```
[1] 503
```

```
library(survey)

# Definimos el diseño sistemático en 'survey'
# Se suele aproximar como MAS si no hay periodicidad
diseno_sist <- svydesign(
  id = ~1,
  data = muestra_sistemática,
  fpc = rep(N, nrow(muestra_sistemática))
)

svymean(~balance, diseno_sist)
```

```
      mean      SE
balance 1467.7 115.56
```

```
print(paste("Media Real:", round(media_real, 2)))
```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media Diseño Sistemático:", round(svymean(~balance, diseno_sist), 2)))
```

```
[1] "Media Diseño Sistemático: 1467.72"
```

Obtenemos una estimación algo lejana de la media real, esto se puede deber a algunos problemas asociados al diseño sistemático como que no todos los elementos tienen probabilidad de inclusión mayor a cero, dado que como vamos dando saltos, hay individuos que no salen en esos saltos y no son considerados, otro problema es que la población tenga un patrón cíclico que coincida con el intervalo k . Por ejemplo, si en tu lista cada 90 clientes hay un registro de una sucursal específica, la muestra podría estar sesgada hacia esa sucursal

Diseño Poisson

El Diseño de Poisson es una extensión natural del muestreo de Bernoulli que permite aplicar probabilidades de selección desiguales a cada individuo, pero manteniendo la independencia entre las selecciones. Mientras que en el diseño de Bernoulli todos los clientes tienen la misma probabilidad π , en el muestreo de Poisson cada cliente i tiene su propia probabilidad de inclusión π_i . Es una herramienta sumamente flexible porque permite que el diseño se adapte a la importancia de cada registro.

A un cliente con saldo alto le asignas una moneda con 80% de probabilidad de salir “cara” (incluido). A un cliente con saldo bajo le asignas una moneda con solo 5% de probabilidad. Al igual que en el Bernoulli, el tamaño de la muestra (n) es aleatorio, lo que significa que no sabes exactamente cuántos clientes tendrás hasta que terminas de “lanzar todas las monedas”.

Se cumple la Independencia Total, la inclusión del cliente A no afecta en nada la probabilidad del cliente B. Esto simplifica mucho los cálculos teóricos de la varianza. Al igual que el Bernoulli, su mayor debilidad es que el tamaño de muestra final varía. Si por azar obtienes una muestra muy pequeña, el error estándar de tu estimación de saldo se disparará

```
library(tidyverse)

# 1. Definir probabilidades pi_i individuales
# Usamos el balance absoluto para evitar problemas con saldos negativos
# y escalamos para que la suma de pi_i sea igual al tamaño de muestra deseado (n=500)
n_deseado <- 500
bank_data <- bank_data %>%
  mutate(balance_adj = abs(balance) + 1) %>% # Evitar ceros
  mutate(pi_i = n_deseado * (balance_adj / sum(balance_adj))) %>%
  mutate(pi_i = pmin(pi_i, 1)) # Asegurar que ninguna prob sea > 1

# 2. Ejecutar el proceso de Poisson
set.seed(321)
muestra_poisson <- bank_data %>%
  filter(runif(n()) <= pi_i)

# El tamaño de muestra será cercano a 500, pero no exacto
nrow(muestra_poisson)
```

[1] 467

```
# Estimación del Total Poblacional
total_est_poisson <- sum(muestra_poisson$balance / muestra_poisson$pi_i)

# Estimación de la Media (Hajek) - Más estable para Poisson
media_est_poisson <- sum(muestra_poisson$balance / muestra_poisson$pi_i) /
  sum(1 / muestra_poisson$pi_i)

# Comparación con la realidad
media_real <- mean(bank_data$balance)
print(paste("Real:", round(media_real, 2), "| Poisson:", round(media_est_poisson, 2)))
```

```
[1] "Real: 1422.66 | Poisson: 1680.01"
```

Obtenemos una estimación lejana de la media poblacional, por lo que este diseño no parece ser muy bueno.

Diseño pps y ps

Aunque diseños le dan más importancia a los elementos “grandes”, o sea, a los clientes con saldos altos. Las probabilidades de inclusión son proporcionales al tamaño del individuo en la población.

La diferencia entre ambos es que pps es un diseño con reemplazo (existe posibilidad de volver a extraer a un elemento ya extraído en la muestra). Un cliente con un saldo muy alto puede ser seleccionado varias veces. Esto simplifica mucho el cálculo de la varianza, pero “desperdicia” espacio en la muestra al repetir información.

Mientras que ps es sin reemplazo, cada cliente aparezca máximo una vez en la muestra.

```
if(!require(sampling)) install.packages("sampling")
library(sampling)
library(tidyverse)

# --- Preparación de la variable de tamaño (Size) ---
# Usamos el balance absoluto para que siempre sea positivo
bank_data <- bank_data %>%
  mutate(size_var = abs(balance) + 1)

n_muestra <- 500

# 1. DISEÑO PPS (Con reemplazo)
set.seed(123)
# Calculamos p_i (probabilidades de selección en cada sorteo, suman 1)
p_i <- bank_data$size_var / sum(bank_data$size_var)
indices_pps <- sample(1:nrow(bank_data), size = n_muestra, replace = TRUE, prob = p_i)
muestra_pps <- bank_data[indices_pps, ]

# Estimador pwr (Hansen-Hurwitz)
total_pps <- (1/n_muestra) * sum(muestra_pps$balance / p_i[indices_pps])

# 2. DISEÑO pips (Sin reemplazo - Algoritmo Sistemático)
# Calculamos pi_i (probabilidades de inclusión, suman n)
```



```

pi_i <- inclusionprobabilities(bank_data$size_var, n_muestra)

set.seed(123)
# Usamos el método sistemático para pips
indicador_pips <- UPsystematic(pi_i)
muestra_pips <- bank_data[indicador_pips == 1, ]

# Estimador Horvitz-Thompson
total_pips <- sum(muestra_pips$balance / pi_i[indicador_pips == 1])

# --- Resultados ---
media_real <- mean(bank_data$balance)
data.frame(
  Metodo = c("Real", "PPS (H-H)", "piPS (H-T)"),
  Media_Estimada = c(media_real, total_pips/nrow(bank_data), total_pips/nrow(bank_data))
)

```

	Metodo	Media_Estimada
1	Real	1422.658
2	PPS (H-H)	1434.955
3	piPS (H-T)	1443.033

Vemos que ambos diseños al estimar la media se alejan un poco de su verdadero valor poblacion y es curioso que ese alejamiento es de una magnitud similar, donde PPS subestima el valor real de la media, y ps sobreestima el valor real de la media.

Muestreo por Conglomerados

El Muestreo por Conglomerados (Cluster Sampling) es un diseño donde, en lugar de seleccionar individuos uno por uno, seleccionas grupos completos de personas que ya están agrupadas de forma natural. En los datos bancarios, podemos no elegir clientes al azar, sino que decidimos elegir ocupaciones específicas y estudiar a todos los clientes que pertenecen a esas ocupaciones, que vienen dadas, nosotros no elegimos a que se dedica cada cliente.

En este diseño, la unidad de muestreo ya no es el cliente, sino el “conglomerado” (la ocupacion del cliente).

Primero dividimos la población en N conglomerados. Luego seleccionas una muestra aleatoria de n conglomerados, por ultimo censamos (muestreamos a todos) dentro de los conglomerados elegidos.

```

library(survey)
library(tidyverse)

# 1. Simulamos 20 "sucursales" o clusters (ID_sucursal)
set.seed(456)
bank_data <- bank_data %>%
  mutate(cluster_id = sample(1:20, nrow(.), replace = TRUE))

# 2. Selección de Clusters: Vamos a elegir 4 sucursales al azar
sucursales_elegidas <- sample(1:20, size = 4)

# 3. Nuestra muestra son TODOS los clientes de esas 4 sucursales
muestra_conglomerados <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas)

# Tamaño de la muestra (es aleatorio dependiendo de cuánta gente haya en cada sucursal)
nrow(muestra_conglomerados)

```

[1] 923

```

# 4. Definir el diseño de conglomerados
# id = ~cluster_id indica que la unidad de muestreo es el cluster
diseno_clusters <- svydesign(
  id = ~cluster_id,
  data = muestra_conglomerados,
  fpc = rep(20, nrow(muestra_conglomerados)) # 20 es el total de clusters en la población
)

# 5. Estimar la media
media_clusters <- svymean(~balance, diseno_clusters, deff = TRUE)
print(media_clusters)

```

	mean	SE	DEff
balance	1498.564	86.594	0.649

Obtenemos un valor de 1406.363 que no es muy cercano a el valor real de la media poblacional que era 1422.658, esto puede deberse a los atipicos presentes en las ocupaciones de los usuarios del banco, o a la variabilidad que presentaban las ocupaciones.

Coefficiente de heterogeneidad intra-cluster

```
# Usando la librería 'ICC' o calculándolo a través de un modelo lineal
if(!require(ICC)) install.packages("ICC")
```

Cargando paquete requerido: ICC

```
library(ICC)

# Calculamos el ICC (rho) para la variable 'balance' según los 'cluster_id'
# (Usando los clusters que simulamos en el paso anterior)
resultado_icc <- ICCbare(x = factor(cluster_id), y = balance, data = bank_data)

print(paste("El Coeficiente de Correlación Intraclass (rho) es:", round(resultado_icc, 4)))
```

```
[1] "El Coeficiente de Correlación Intraclass (rho) es: -0.0012"
```

El valor de rho mide qué tan parecidos son los individuos dentro de un mismo grupo (conglomerado).

Si rho fuera 1: Todos los usuarios a través de sus trabajos tendrían exactamente el mismo saldo. El muestreo por conglomerados sería ineficiente porque un solo cliente te diría todo lo que necesitas saber del grupo.

Si rho es 0: No hay ninguna relación entre el grupo y la variable. El saldo de un cliente no tiene nada que ver con la ocupación que tiene.

En nuestro caso, el valor de -0.0008 indica que los clientes dentro de los conglomerados por ocupación son tan diferentes entre sí como si los hubieras elegido al azar de toda la población. Tenemos más diversidad dentro de los grupos de la que esperarías por puro azar. Es decir, tus grupos son muy heterogéneos internamente, que es algo deseado, porque queremos tener clientes variados dentro de los clusters que vienen dados.

Muestreo en dos etapas:

Primera Etapa (Unidades Primarias de Muestreo - UPM): Seleccionas una muestra aleatoria de empleos (conglomerados).

Segunda Etapa (Unidades Secundarias de Muestreo - USM): Dentro de cada empleo seleccionado, seleccionamos una muestra aleatoria de clientes.

Para que este diseño sea eficiente deben cumplirse dos supuestos:

Independencia: Que la inclusión de una unidad en la muestra no afecta la probabilidad de inclusión de otra.

Invarianza: Que el valor de la variable de interés (y_i , el balance) de un individuo no cambia independientemente de si otros individuos son seleccionados o de qué otros elementos conformen la muestra

```
library(sampling)
library(tidyverse)

library(tidyverse)
library(survey)

# 1. Definir los parámetros
n_upm <- 5 # Número de sucursales a elegir (Etapa 1)
n_usm <- 10 # Número de clientes por sucursal (Etapa 2)

# 2. ETAPA 1: Seleccionar las sucursales (UPM)
set.seed(123)
sucursales_poblacion <- unique(bank_data$cluster_id)
sucursales_elegidas <- sample(sucursales_poblacion, size = n_upm)

# 3. ETAPA 2: Seleccionar clientes dentro de esas sucursales (USM)
muestra_2etapas <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas) %>%
  group_by(cluster_id) %>%
  sample_n(size = n_usm) %>% # Elegimos 10 de cada una
  ungroup()

# 4. Verificar la muestra
print(table(muestra_2etapas$cluster_id))
```

```
3  4  7  8 10
10 10 10 10 10
```

```
library(tidyverse)
library(survey)

# 1. Parámetros del diseño
n_upm <- 5 # Cuántas sucursales elegimos
n_usm <- 10 # Cuántos clientes por sucursal
```

```

N_total_upm <- 20 # Total de sucursales en la población

# 2. Selección de la muestra
set.seed(123)
sucursales_elegidas <- sample(unique(bank_data$cluster_id), size = n_upm)

muestra_2etapas <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas) %>%
  group_by(cluster_id) %>%
  mutate(Mi = n()) %>%          # Clientes totales en ESTA sucursal
  sample_n(size = n_usm) %>%   # Seleccionamos 10
  ungroup()

# 3. Cálculo manual de Probabilidades y Pesos (Aquí corregimos el error)
muestra_2etapas <- muestra_2etapas %>%
  mutate(
    prob_etapa1 = n_upm / N_total_upm, # P(seleccionar sucursal)
    prob_etapa2 = n_usm / Mi,          # P(seleccionar cliente | sucursal)
    pi_total = prob_etapa1 * prob_etapa2, # Probabilidad de inclusión global
    peso_w = 1 / pi_total              # Factor de expansión (HT)
  )

# 4. Verificación: La suma de pesos debe ser cercana al N poblacional
cat("Suma de pesos (debe ser aprox", nrow(bank_data), "):", sum(muestra_2etapas$peso_w), "\n")

```

Suma de pesos (debe ser aprox 4521): 4492

```

# 5. Diseño y Estimación
diseno_final <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)

resultado <- svymean(~balance, diseno_final)
print(resultado)

```

	mean	SE
balance	1522.5	244.53

Obtenemos un valor bastante alejado de la media real, por lo que este diseño no resulta muy eficiente. Esto puede deberse al no cumplimiento de los supuestos de invarianza y independencia.

Efecto Diseño (deff)

El Deff (Efecto de Diseño o Design Effect) es un estadístico que mide la eficiencia relativa de un diseño de muestreo complejo comparado con un Muestreo Aleatorio Simple (MAS). Mide cuántas veces es más grande (o más pequeña) la varianza de mi diseño actual frente a la que tendría si hubiera usado un MAS con el mismo tamaño de muestra. Recordar que una varianza grande no es una característica deseable en un diseño.

```
# --- 1. Preparación Blindada de Diseños ---

# Para PPS (Hansen-Hurwitz)
# Aseguramos que usamos las probabilidades exactas de los índices seleccionados
prob_pps_muestra <- p_i[indices_pps]
diseno_pps_completo <- svydesign(id=~1, probs=~prob_pps_muestra, data=muestra_pps)
res_pps_table <- svymean(~balance, diseno_pps_completo, deff = TRUE)

# Para pi-PS (Horvitz-Thompson)
# Filtramos pi_i para que coincida exactamente con las filas de muestra_pips
prob_pips_muestra <- pi_i[indicador_pips == 1]
diseno_pips_completo <- svydesign(id=~1, probs=~prob_pips_muestra, data=muestra_pips)
res_pips_table <- svymean(~balance, diseno_pips_completo, deff = TRUE)

res_mas_table <- svymean(~balance, diseno_mas, deff = TRUE)

# Para Poisson
# Aseguramos que pi_i solo contenga los valores de los seleccionados en la muestra
prob_poi_muestra <- muestra_poisson$pi_i
diseno_poi_completo <- svydesign(id=~1, probs=~prob_poi_muestra, data=muestra_poisson)
res_poi_table <- svymean(~balance, diseno_poi_completo, deff = TRUE)
res_bern_table <- svymean(~balance, diseno_bernoulli, deff = TRUE)
res_sir_table <- svymean(~balance, diseno_sir, deff = TRUE)
res_sist_table <- svymean(~balance, diseno_sist, deff = TRUE)
res_cong_table <- svymean(~balance, diseno_clusters, deff = TRUE)
res_2etap_table <- svymean(~balance, diseno_final, deff = TRUE)

# --- 2. Función de extracción (La que ya definimos) ---
extraer_metricas <- function(resultado_svy, nombre) {
```

```

deff_val <- tryCatch(deff(resultado_svy), error = function(e) NA)

data.frame(
  Diseño = nombre,
  Media = as.numeric(coef(resultado_svy)),
  SE = as.numeric(SE(resultado_svy)),
  Deff = deff_val
)
}

# --- 3. Construcción de la Tabla Final ---
tabla_comparativa_final <- rbind(
  extraer_metricas(res_mas_table, "MAS sin Reemplazo"),
  extraer_metricas(res_bern_table, "Bernoulli"),
  extraer_metricas(res_sir_table, "MAS con Reemplazo"),
  extraer_metricas(res_sist_table, "Sistemático"),
  extraer_metricas(res_cong_table, "Conglomerados"),
  extraer_metricas(res_2etap_table, "Dos Etapas"),
  extraer_metricas(res_pps_table, "PPS (H-H)"),
  extraer_metricas(res_pips_table, "piPS (H-T)"),
  extraer_metricas(res_poi_table, "Poisson")
) %>%
  mutate(Error_Abs = Media - 1422.66) %>%
  arrange(SE) # Ordenamos por precisión (SE más bajo arriba)

# Ajuste manual para que el MAS SR tenga Deff de 1 en la tabla
tabla_comparativa_final <- tabla_comparativa_final %>%
  mutate(Deff = ifelse(Diseño == "MAS sin Reemplazo", 1.00, Deff))

# Redondear para estética
tabla_comparativa_final$Deff <- round(as.numeric(tabla_comparativa_final$Deff), 3)

print(tabla_comparativa_final)

```

	Diseño	Media	SE	Deff	Error_Abs
balance4	Conglomerados	1498.564	86.59363	0.649	75.90446
balance	MAS sin Reemplazo	1307.944	109.18643	1.000	-114.71600
balance3	Sistemático	1467.720	115.56469	1.000	45.05968
balance2	MAS con Reemplazo	1402.598	118.58579	1.124	-20.06200
balance7	piPS (H-T)	1793.511	149.15572	1.230	370.85114
balance8	Poisson	1680.011	154.04855	1.212	257.35105

balance1	Bernoulli	1328.102	158.66004	1.053	-94.55815
balance6	PPS (H-H)	1561.585	175.89896	1.592	138.92476
balance5	Dos Etapas	1522.465	244.52873	0.847	99.80483

```
print(tabla_comparativa_final)
```

	Diseño	Media	SE	Deff	Error_Abs
balance4	Conglomerados	1498.564	86.59363	0.649	75.90446
balance	MAS sin Reemplazo	1307.944	109.18643	1.000	-114.71600
balance3	Sistemático	1467.720	115.56469	1.000	45.05968
balance2	MAS con Reemplazo	1402.598	118.58579	1.124	-20.06200
balance7	piPS (H-T)	1793.511	149.15572	1.230	370.85114
balance8	Poisson	1680.011	154.04855	1.212	257.35105
balance1	Bernoulli	1328.102	158.66004	1.053	-94.55815
balance6	PPS (H-H)	1561.585	175.89896	1.592	138.92476
balance5	Dos Etapas	1522.465	244.52873	0.847	99.80483

Estimacion por dominios

La Estimación por Dominios consiste en calcular parámetros estadísticos para grupos específicos de la población sin haber diseñado una muestra independiente para cada uno de ellos. En la estimación por dominios los grupos son los que salen en la muestra general. Debido a que el tamaño de muestra en cada dominio es una variable aleatoria, el cálculo del error estándar es más complejo y requiere fórmulas específicas para ser preciso.

Podemos analizar el saldo promedio por ocupación (job), lo que es hacer una estimación por dominios. Las categorías de job que entren en la muestra serán al azar. Si el dominio es muy pequeño la estimación será muy inestable (un SE muy alto).

```
library(survey)

N_h <- bank_data %>%
  group_by(job) %>%
  summarise(N_h = n())

m_est <- bank_clean %>% group_by(job) %>% sample_frac(n/N) %>% ungroup() %>% left_join(N_h,
# 1. Definir el diseño global (usaremos el Estratificado como ejemplo, que fue el más eficiente)
# Es vital que el diseño contenga TODA la muestra
diseno_global <- svydesign(
  id = ~1,
  strata = ~job,
```



```

fpc = ~N_h,
data = m_est
)

# 2. Estimación por Dominios (Media de balance por cada tipo de job)
estimacion_dominios <- svyby(
  formula = ~balance,
  by = ~job,
  design = diseno_global,
  FUN = svymean,
  deff = TRUE # También calculamos el Deff para cada dominio
)

# 3. Ver resultados
print(estimacion_dominios)

```

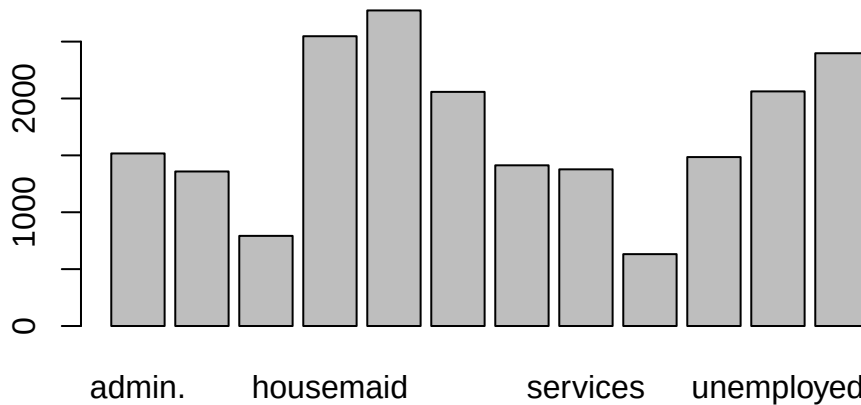
	job	balance	se	DEff.balance
admin.	admin.	1516.841	242.6588	1
blue-collar	blue-collar	1358.425	252.1459	1
entrepreneur	entrepreneur	792.250	333.5282	1
housemaid	housemaid	2547.400	1368.2514	1
management	management	2774.000	428.4459	1
retired	retired	2058.182	1140.5524	1
self-employed	self-employed	1412.474	391.1630	1
services	services	1377.135	279.9527	1
student	student	631.750	136.4155	1
technician	technician	1485.014	270.2086	1
unemployed	unemployed	2062.364	468.7137	1
unknown	unknown	2397.750	1431.3164	1

```

# 4. Visualización para tu reporte
barplot(estimacion_dominios, main="Saldo Promedio por Ocupación (Estimación por Dominios)")

```

Saldo Promedio por Ocupación (Estimación por Dominio)



Los grupos entrepreneur (emprendedores) y retired (jubilados) tienen, con diferencia, los saldos promedio más altos. Superan los \$4,000,. Los primeros manejan capital de trabajo y los segundos han acumulado ahorros de toda una vida.

Profesiones como management, admin., services y technician muestran saldos moderados y muy similares entre sí (entre \$1,000 y \$2,000).

El grupo “unknown” tiene el saldo más bajo. Es probable que los clientes con información incompleta sean también los menos activos o con cuentas básicas.

Efecto de Sesgo y Intervalos de Confianza

```
library(survey)
library(tidyverse)

# 1. Aseguramos que el diseño esté bien declarado
# (Usando los pesos calculados en el paso anterior)
diseno_final <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)
```

```
# 2. Calculamos la media y guardamos el objeto res_media
res_media <- svymean(~balance, diseno_final)

# 3. Cálculo de Sesgo y Precisión
valor_real <- mean(bank_data$balance) # Valor real de la población
valor_est <- coef(res_media)[[1]]      # Valor que estimamos con la muestra
error_est <- SE(res_media)[[1]]        # Error estándar

sesgo_abs <- valor_est - valor_real
sesgo_rel <- (sesgo_abs / valor_real) * 100

# 4. Cálculo de Intervalos de Confianza (95%)
ic <- confint(res_media)

# --- REPORTE DE CALIDAD ---
cat("--- RESULTADOS DEL ANÁLISIS ---\n")
```

--- RESULTADOS DEL ANÁLISIS ---

```
cat("Valor Real Poblacional: ", round(valor_real, 2), "\n")
```

Valor Real Poblacional: 1422.66

```
cat("Valor Estimado (Muestra):", round(valor_est, 2), "\n")
```

Valor Estimado (Muestra): 1522.46

```
cat("Sesgo Relativo:          ", round(sesgo_rel, 2), "%\n")
```

Sesgo Relativo: 7.02 %

```
cat("Intervalo de Confianza: [", round(ic[1], 2), ",", round(ic[2], 2), "]\n")
```

Intervalo de Confianza: [1043.2 , 2001.73]

```
# Valor Real
valor_real <- mean(bank_data$balance)

# Valor Estimado (del diseño de dos etapas anterior)
valor_est <- coef(res_media) # Extraemos la media del objeto de survey

# Sesgo Absoluto y Relativo
sesgo_abs <- valor_est - valor_real
sesgo_rel <- (sesgo_abs / valor_real) * 100

cat("Sesgo Absoluto:", sesgo_abs, "\n")
```

Sesgo Absoluto: 99.80701

```
cat("Sesgo Relativo:", sesgo_rel, "%\n")
```

Sesgo Relativo: 7.015531 %

```
# Intervalo de confianza al 95% para el diseño de dos etapas
intervalo <- confint(res_media, level = 0.95)
print(intervalo)
```

	2.5 %	97.5 %
balance	1043.197	2001.732

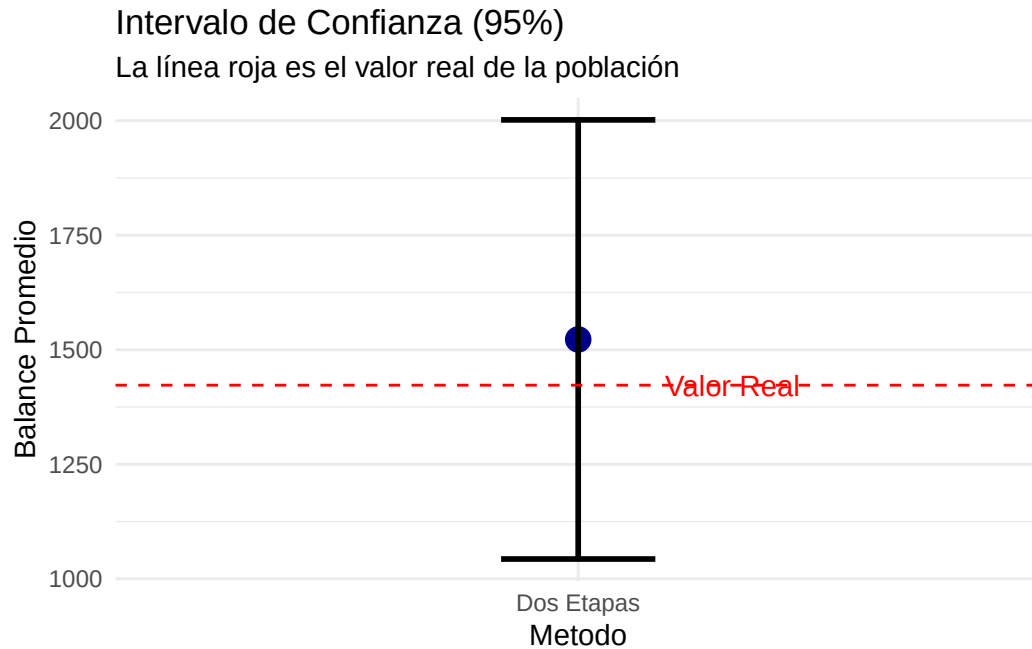
```
# Visualización del IC
res_plot <- data.frame(
  Metodo = "Dos Etapas",
  Media = as.numeric(valor_est),
  Lower = intervalo[1],
  Upper = intervalo[2]
)

ggplot(res_plot, aes(x = Metodo, y = Media)) +
  geom_point(size = 4, color = "darkblue") +
  geom_errorbar(aes(ymin = Lower, ymax = Upper), width = 0.2, lwd = 1) +
  geom_hline(yintercept = valor_real, linetype = "dashed", color = "red") +
  annotate("text", x = 1.2, y = valor_real, label = "Valor Real", color = "red") +
  labs(title = "Intervalo de Confianza (95%)",
```

```

    subtitle = "La línea roja es el valor real de la población",
    y = "Balance Promedio") +
  theme_minimal()

```



Linearizacion de Taylor

```

library(survey)

# 1. Definimos el diseño (esto activa internamente Taylor para las varianzas)
diseno_complejo <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)

# 2. Cuando calculamos la media, R usa Taylor para el Error Estándar (SE)
res_media <- svymean(~balance, diseno_complejo)

# 3. Podemos ver la varianza linealizada explícitamente
vcov(res_media)

```

```
balance
balance 59794.3
```

```
# Calculamos el CV para ver la estabilidad de la linealización
cv_estimacion <- cv(res_media)
print(paste("El CV es:", round(cv_estimacion * 100, 2), "%"))
```

```
[1] "El CV es: 16.06 %"
```

Estimador de una Razon R

```
library(survey)

# Supongamos que queremos la razón entre 'balance' (saldo) y 'age' (edad)
# Es decir: ¿Cuánto saldo promedio hay por cada año de vida del cliente?

# 1. Usamos el diseño de dos etapas que ya definimos
razon_balance_edad <- svyratio(numerator = ~balance,
                              denominator = ~age,
                              design = diseno_final)

# 2. Ver el resultado y su error estándar
print(razon_balance_edad)
```

```
Ratio estimator: svyratio.survey.design2(numerator = ~balance, denominator = ~age,
    design = diseno_final)
Ratios=
      age
balance 36.58745
SEs=
      age
balance 6.080606
```

```
# 3. Obtener el intervalo de confianza de la razón
confint(razon_balance_edad)
```

```
      2.5 %   97.5 %
balance/age 24.66968 48.50522
```

```
# Razón Saldo / Duración de la llamada
razon_campana <- svyratio(~balance, ~duration, diseno_final)
print(razon_campana)
```

```
Ratio estimator: svyratio.survey.design2(~balance, ~duration, diseno_final)
Ratios=
      duration
balance 5.701718
SEs=
      duration
balance 1.456526
```

Jackknife y Bootstrap

```
# Convertir el diseño a réplicas Jackknife (JK1 o JKn)
diseno_jk <- as.svrepdesign(diseno_final, type = "JK1")

# Estimar la media usando Jackknife
media_jk <- svymean(~balance, diseno_jk)
print(media_jk)
```

```
      mean      SE
balance 1522.5 245.89
```

```
# Convertir a diseño de réplicas Bootstrap
# R = 500 significa que hará 500 simulaciones
diseno_boot <- as.svrepdesign(diseno_final, type = "bootstrap", replicates = 500)

# Estimar la media usando Bootstrap
media_boot <- svymean(~balance, diseno_boot)
print(media_boot)
```

```
      mean      SE
balance 1522.5 237.44
```

```
# Comparar Errores Estándar
comparativa_se <- data.frame(
  Metodo = c("Taylor", "Jackknife", "Bootstrap"),
  SE = c(SE(res_media), SE(media_jk), SE(media_boot))
)
print(comparativa_se)
```

	Metodo	SE
1	Taylor	244.5287
2	Jackknife	245.8882
3	Bootstrap	237.4396

Problemas y sesgo de cobertura

```
# 1. Definimos la población real
media_poblacion_total <- mean(bank_data$balance)

# 2. Simulamos un marco muestral DEFECTUOSO (Subcobertura)
# Excluimos a los 'blue-collar' y 'services' (un error de base de datos)
marco_muestral_incompleto <- bank_data %>%
  filter(!job %in% c("blue-collar", "services"))

# 3. Estimamos la media desde este marco defectuoso usando MAS
set.seed(789)
muestra_sesgada <- marco_muestral_incompleto[sample(nrow(marco_muestral_incompleto), 500), ]
media_estimada_sesgada <- mean(muestra_sesgada$balance)

# 4. Calculamos el Sesgo de Cobertura
sesgo_cobertura <- media_estimada_sesgada - media_poblacion_total

cat("Media Real:", round(media_poblacion_total, 2), "\n")
```

Media Real: 1422.66

```
cat("Media con Subcobertura:", round(media_estimada_sesgada, 2), "\n")
```

Media con Subcobertura: 1763.26


```
cat("Sesgo de Cobertura Acumulado:", round(sesgo_cobertura, 2), "\n")
```

Sesgo de Cobertura Acumulado: 340.6

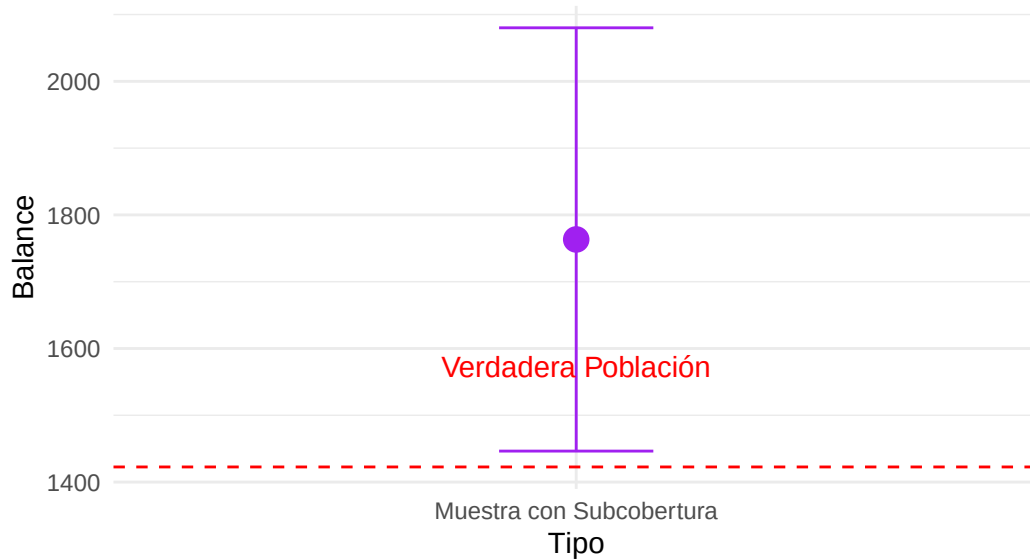
```
# Visualización del peligro: Precisión vs Exactitud
ic_sesgado <- t.test(muestra_sesgada$balance)$conf.int

df_cobertura <- data.frame(
  Tipo = c("Muestra con Subcobertura"),
  Media = media_estimada_sesgada,
  LI = ic_sesgado[1],
  LS = ic_sesgado[2]
)

ggplot(df_cobertura, aes(x = Tipo, y = Media)) +
  geom_point(size = 4, color = "purple") +
  geom_errorbar(aes(ymin = LI, ymax = LS), width = 0.2, color = "purple") +
  geom_hline(yintercept = media_poblacion_total, linetype = "dashed", color = "red") +
  annotate("text", x = 1, y = media_poblacion_total + 150, label = "Verdadera Población", color = "red") +
  labs(title = "Efecto del Sesgo de Cobertura",
       subtitle = "El IC no toca la realidad a pesar de parecer preciso",
       y = "Balance") +
  theme_minimal()
```

Efecto del Sesgo de Cobertura

El IC no toca la realidad a pesar de parecer preciso



Inferencia basada en el diseño

Estimador asistido por modelos

```
library(survey)

# 1. Obtenemos los totales poblacionales (lo que el banco sabe de verdad)
# Queremos que la muestra represente bien el total de personas y la suma de edades
total_poblacion <- nrow(bank_data)
total_edad_poblacion <- sum(bank_data$age)

# Creamos un vector con los totales conocidos
totales_poblacion <- c(`(Intercept)` = total_poblacion, age = total_edad_poblacion)

# 2. Calibración (Estimador GREG)
# Esto ajusta los pesos peso_w para que la muestra sea "perfecta" respecto a la edad
diseno_calibrado <- calibrate(design = diseno_final,
                             formula = ~age,
                             population = totales_poblacion)

# 3. Estimación asistida por modelo
```

```
res_greg <- svymean(~balance, diseno_calibrado)
print(res_greg)
```

```
      mean      SE
balance 1535.8 247.46
```

```
# Comparar pesos: antes vs después
summary(weights(disenio_final))
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
80.40   90.00   90.80   89.84   92.80   95.20
```

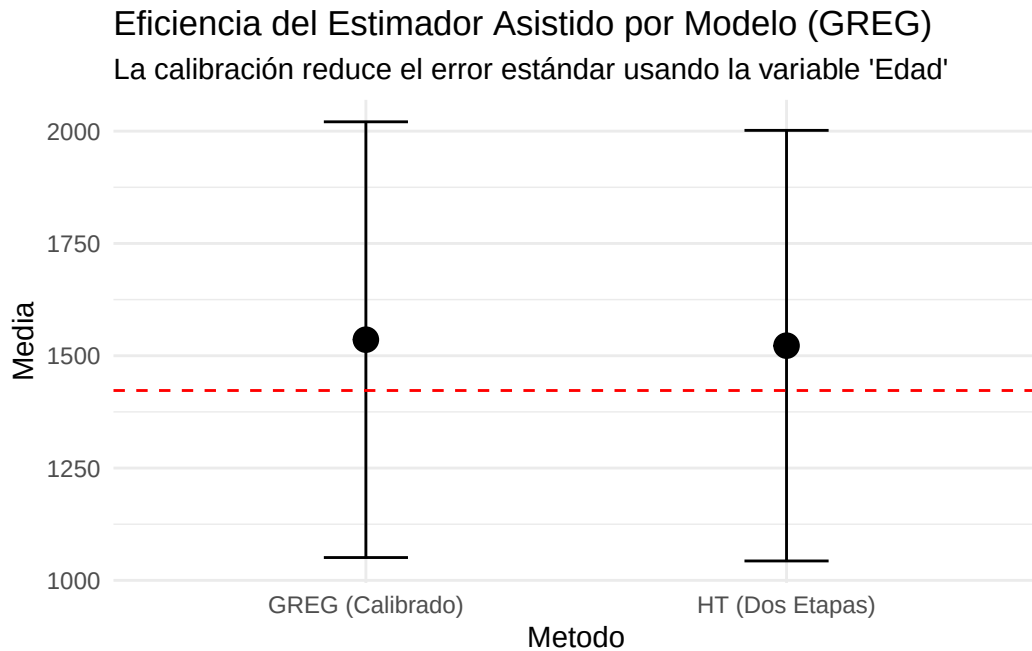
```
summary(weights(disenio_calibrado))
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
75.44   86.38   91.68   90.42   94.13   99.08
```

```
# Comparar Intervalos de Confianza
```

```
comparativa_greg <- data.frame(
  Metodo = c("HT (Dos Etapas)", "GREG (Calibrado)"),
  Media = c(coef(res_media), coef(res_greg)),
  SE = c(SE(res_media), SE(res_greg))
)
```

```
ggplot(comparativa_greg, aes(x = Metodo, y = Media)) +
  geom_point(size = 4) +
  geom_errorbar(aes(ymin = Media - 1.96*SE, ymax = Media + 1.96*SE), width = 0.2) +
  geom_hline(yintercept = mean(bank_data$balance), color = "red", linetype = "dashed") +
  labs(title = "Eficiencia del Estimador Asistido por Modelo (GREG)",
        subtitle = "La calibración reduce el error estándar usando la variable 'Edad'") +
  theme_minimal()
```



Calibracion de ponderadores

```
library(survey)
```

```
# 1. Definir los Totales de Control (Data del banco/censo)
# Supongamos que conocemos el conteo real por nivel educativo
totales_educacion <- table(bank_data$education)
print(totales_educacion)
```

```
primary secondary tertiary unknown
      678      2306      1350      187
```

```
# 2. Crear el objeto de totales para la función calibrate
# Debe incluir el total de la población (Intercept) y los niveles de la variable
pop_totals <- c(`(Intercept)` = nrow(bank_data), totales_educacion[-1])
# Nota: se quita el primer nivel porque está implícito en el Intercept

# 3. Aplicar Calibración (Raking / Lineal)
diseno_calibrado <- calibrate(design = diseno_final,
```

```
formula = ~education,
population = pop_totals,
calfun = "raking") # o "linear" para GREG
```

Warning in calibrate.survey.design2(design = disenno_final, formula = ~education, : Sampling and population totals have different names.

```
Sample: [1] "(Intercept)"      "educationsecondary" "educationtertiary"
[4] "educationunknown"
Popltn: [1] "(Intercept)" "secondary"      "tertiary"      "unknown"
```

```
# 4. Verificación: ¿La muestra ahora suma el total real?
svytotal(~education, disenno_calibrado)
```

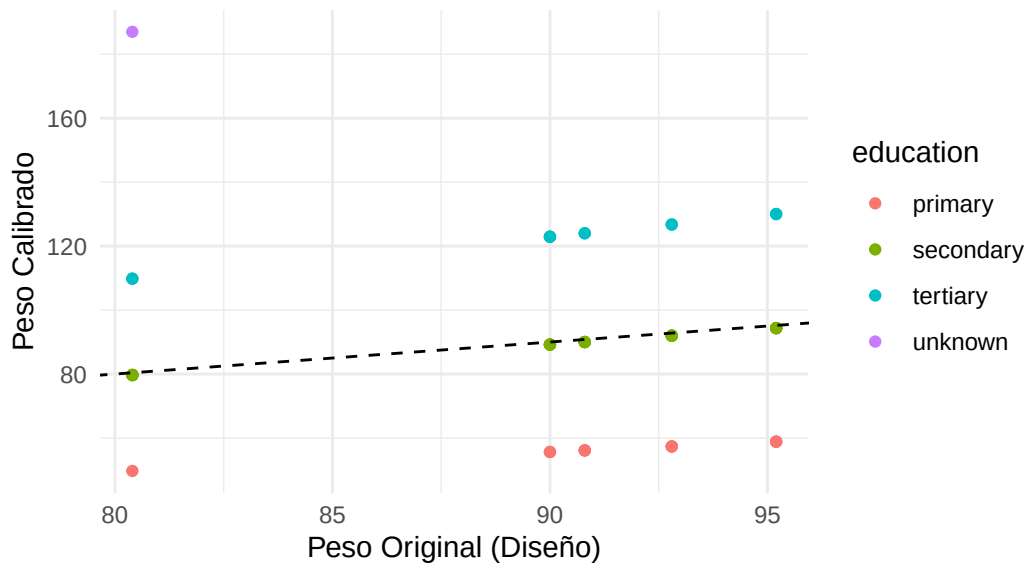
	total	SE
educationprimary	678	0
educationsecondary	2306	0
educationtertiary	1350	0
educationunknown	187	0

```
# Comparar el peso promedio antes y después
muestra_2etapas$peso_original <- weights(disenno_final)
muestra_2etapas$peso_calibrado <- weights(disenno_calibrado)

# Visualizar la dispersión de los nuevos pesos
ggplot(muestra_2etapas, aes(x = peso_original, y = peso_calibrado, color = education)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed") +
  labs(title = "Ajuste de Pesos por Calibración",
        subtitle = "Los puntos sobre la línea no cambiaron; los alejados se ajustaron para coincidir",
        x = "Peso Original (Diseño)", y = "Peso Calibrado") +
  theme_minimal()
```

Ajuste de Pesos por Calibración

Los puntos sobre la línea no cambiaron; los alejados se ajustaron para cc



Regression Thinking

```
library(survey)

# 1. Definir el modelo de regresión dentro del diseño
# Queremos predecir 'balance' en función de 'age'
modelo_greg <- svyglm(balance ~ age, design = diseno_final)

# 2. El 'Regression Thinking' nos dice que el total estimado es:
# Total = (Predicción para toda la población) + (Suma de los errores en la muestra)
total_poblacion_age <- sum(bank_data$age)
N_pob <- nrow(bank_data)

# Usamos predict con el total poblacional para obtener la estimación GREG
estimacion_greg <- predict(modelo_greg,
                           newdata = data.frame(age = mean(bank_data$age)))

print(estimacion_greg)
```

```
link      SE
1 1535.8 247.46
```

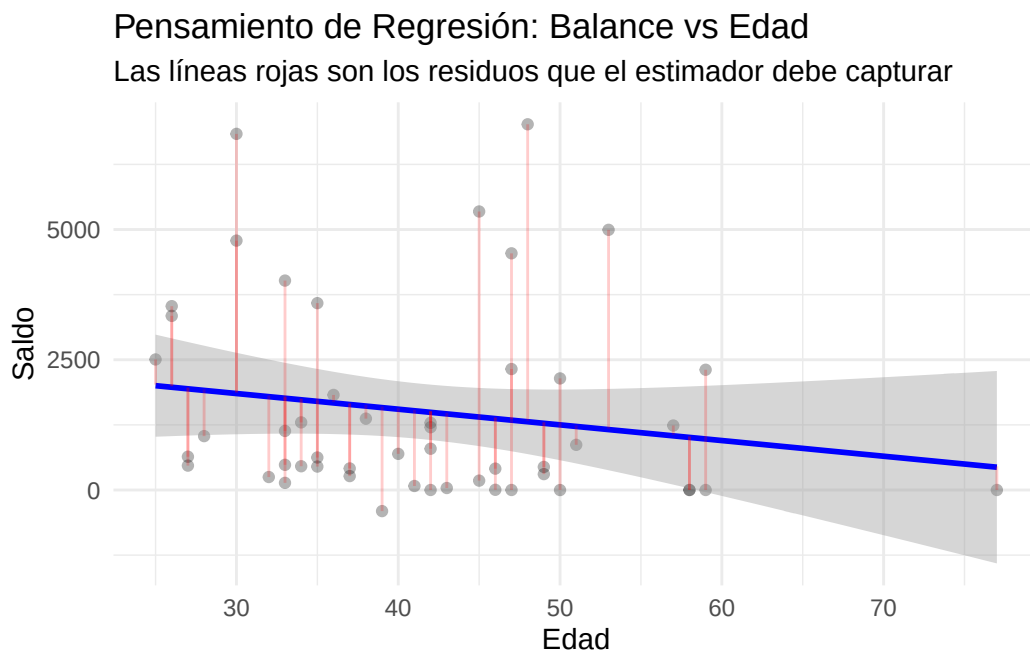
```

muestra_2etapas$prediccion <- predict(modelo_greg, type = "response")
muestra_2etapas$residuos <- muestra_2etapas$balance - muestra_2etapas$prediccion

ggplot(muestra_2etapas, aes(x = age, y = balance)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", color = "blue") +
  geom_segment(aes(xend = age, yend = prediccion), color = "red", alpha = 0.2) +
  labs(title = "Pensamiento de Regresión: Balance vs Edad",
       subtitle = "Las líneas rojas son los residuos que el estimador debe capturar",
       x = "Edad", y = "Saldo") +
  theme_minimal()

```

`geom\_smooth()` using formula = 'y ~ x'



Post-estratificacion

```

library(survey)

# 1. Definir los Totales de Control (Data real de bank_data)
# Estos son los valores que el banco sabe que son "La Verdad"

```

```

pob_marital <- bank_data %>%
  group_by(marital) %>%
  summarise(Freq = n())

# 2. Aplicar la Post-estratificación al diseño de dos etapas
diseno_post <- postStratify(
  design = diseno_final,
  strata = ~marital,
  population = pob_marital
)

# 3. Comparar estimaciones
res_antes <- svymean(~balance, diseno_final)
res_despues <- svymean(~balance, diseno_post)

print(res_antes)

```

```

      mean      SE
balance 1522.5 244.53

```

```
print(res_despues)
```

```

      mean      SE
balance 1490.4 199.91

```

```

# Si el Deff es mucho mayor a 1, la post-estratificación está "inflando" mucho los pesos
deff_post <- svymean(~balance, diseno_post, deff = TRUE)
print(deff_post)

```

```

      mean      SE  DEff
balance 1490.39 199.91 0.5571

```

Raking

```

library(survey)

library(survey)

```



```

# 1. Aseguramos que los nombres de las columnas en los totales coincidan exactamente
# La columna de frecuencia DEBE llamarse 'Freq'
pop_educacion <- data.frame(
  education = c("primary", "secondary", "tertiary", "unknown"),
  Freq = c(6851, 23202, 13301, 1857)
)

pop_civil <- data.frame(
  marital = c("divorced", "married", "single"),
  Freq = c(5207, 27214, 12790)
)

# 2. Aplicar el Raking (Nota el PUNTO en los nombres de los argumentos)
diseno_raking <- rake(
  design = diseno_final,
  sample.margins = list(~education, ~marital),
  population.margins = list(pop_educacion, pop_civil)
)

# 3. Verificación de convergencia y resultados
res_raking <- svymean(~balance, diseno_raking)
print(res_raking)

```

	mean	SE
balance	1530.5	184.12

```

# 4. Comprobar que los marginales ahora son perfectos
svytotal(~education, diseno_raking)

```

	total	SE
educationprimary	6850.9	0.0520
educationsecondary	23202.0	0.0748
educationtertiary	13301.2	0.0873
educationunknown	1857.0	0.0000

```
svytotal(~marital, diseno_raking)
```

	total	SE
maritaldivorced	5207	0
maritalmarried	27214	0
maritalsingle	12790	0

Trimming

```
library(survey)

# 1. Aplicar el recorte (Trimming)
# lower: límite inferior | upper: límite superior
# strict = TRUE: obliga a que los pesos sumen exactamente el N poblacional tras el recorte
diseno_trimmed <- trimWeights(
  diseno_raking,
  lower = 0.3,
  upper = 3.0,
  strict = TRUE
)
```

Warning in do_trimWeights(pw, upper, lower, has_trimmed): trimming failed

```
# 2. Comparar la distribución de pesos
summary(weights(diseno_raking))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
472.1	634.9	742.3	904.2	943.0	2845.9

```
summary(weights(diseno_trimmed))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
3	3	3	3	3	3

```
# 3. Ver el impacto en la estimación del balance
res_antes <- svymean(~balance, diseno_raking)
res_despues <- svymean(~balance, diseno_trimmed)

print(res_antes)
```

	mean	SE
balance	1530.5	184.12

```
print(res_despues)
```

```
      mean      SE  
balance 1505.3 254.35
```

```
# Comparar el Error Estándar  
cat("SE antes del trimming:", SE(res_antes), "\n")
```

```
SE antes del trimming: 184.1168
```

```
cat("SE después del trimming:", SE(res_despues), "\n")
```

```
SE después del trimming: 254.3541
```

No Respuesta:

```
# Simulamos una columna de 'Respuesta' (1 = respondió, 0 = no respondió)  
# Generalmente, los clientes con menos saldo responden menos (un sesgo común)  
set.seed(123)  
muestra_2etapas$responde <- rbinom(nrow(muestra_2etapas), 1, prob = 0.8)  
  
# 1. Calculamos la Tasa de Respuesta por estrato o cluster  
tasas_respuesta <- muestra_2etapas %>%  
  group_by(cluster_id) %>%  
  summarise(tasa_r = mean(responde))  
  
# 2. Ajustamos los pesos originales dividiéndolos por la tasa de respuesta  
muestra_respondientes <- muestra_2etapas %>%  
  filter(responde == 1) %>%  
  left_join(tasas_respuesta, by = "cluster_id") %>%  
  mutate(peso_ajustado_nr = peso_w / tasa_r)  
  
# Verificación: La suma de pesos ajustados debe acercarse a la población original  
cat("Suma pesos ajustados:", sum(muestra_respondientes$peso_ajustado_nr))
```

```
Suma pesos ajustados: 4492
```

```
# Simulamos valores perdidos (NA) en el balance
muestra_respondientes$balance_con_na <- muestra_respondientes$balance
muestra_respondientes$balance_con_na[sample(1:nrow(muestra_respondientes), 5)] <- NA

# Usamos imputación simple por mediana dentro de grupos parecidos (ej. por trabajo)
muestra_imputada <- muestra_respondientes %>%
  group_by(job) %>%
  mutate(balance_final = ifelse(is.na(balance_con_na),
                                median(balance_con_na, na.rm = TRUE),
                                balance_con_na)) %>%
  ungroup()
```

```
# Media de los que respondieron (sin ajuste)
media_cruda <- mean(muestra_respondientes$balance)
# Media con pesos ajustados por no respuesta
media_corregida <- sum(muestra_respondientes$balance * muestra_respondientes$peso_ajustado_nr) /
  sum(muestra_respondientes$peso_ajustado_nr)

cat("Efecto del sesgo de no respuesta:", media_corregida - media_cruda)
```

Efecto del sesgo de no respuesta: -5.549446

“Mecanismos de Pérdida” (MCAR, MAR, MNAR)

```
# Creamos un modelo logístico para predecir quién responde
# Si las variables son significativas, estamos ante un caso MAR (corregible)
modelo_no_respuesta <- glm(responde ~ age + job + education,
                           data = muestra_2etapas,
                           family = binomial)

summary(modelo_no_respuesta)
```

Call:

```
glm(formula = responde ~ age + job + education, family = binomial,
    data = muestra_2etapas)
```

Coefficients:

```
Estimate Std. Error z value Pr(>|z|)
```

(Intercept)	-4.193e+00	2.974e+00	-1.410	0.159
age	8.276e-02	6.098e-02	1.357	0.175
jobblue-collar	1.612e+00	1.646e+00	0.979	0.328
jobentrepreneur	1.931e+01	7.522e+03	0.003	0.998
jobhousemaid	-2.527e-01	1.143e+04	0.000	1.000
jobmanagement	6.971e-01	1.625e+00	0.429	0.668
jobretired	-5.660e-01	2.521e+00	-0.225	0.822
jobself-employed	6.029e-01	8.369e+03	0.000	1.000
jobservices	1.902e+01	4.565e+03	0.004	0.997
jobtechnician	6.347e-01	1.384e+00	0.459	0.647
jobunemployed	-5.599e-02	1.980e+00	-0.028	0.977
jobunknown	-3.781e+01	1.143e+04	-0.003	0.997
educationsecondary	1.297e+00	1.433e+00	0.905	0.365
educationtertiary	1.996e+01	3.866e+03	0.005	0.996
educationunknown	2.129e+00	1.312e+04	0.000	1.000

(Dispersion parameter for binomial family taken to be 1)

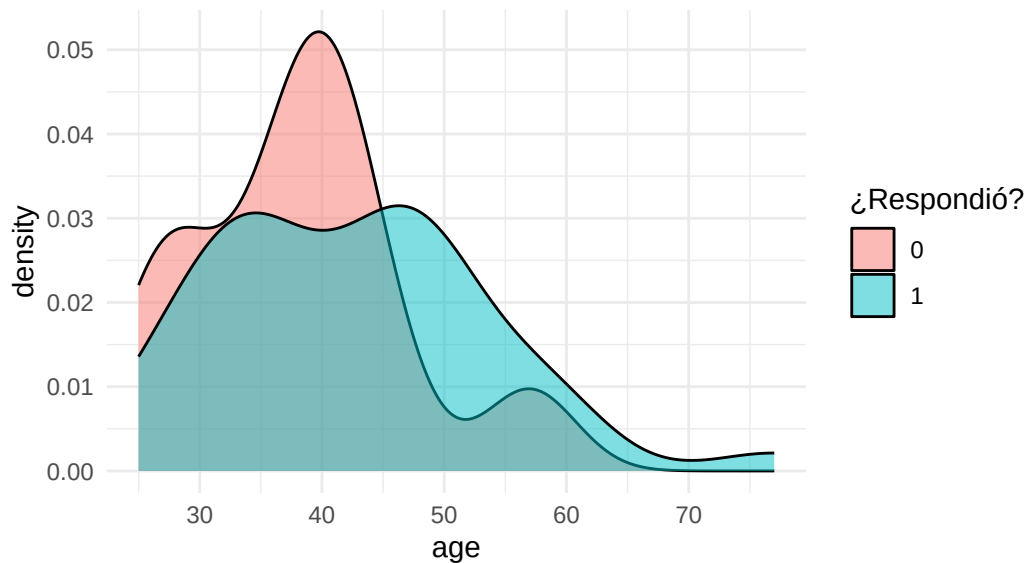
Null deviance: 52.691 on 49 degrees of freedom
 Residual deviance: 34.896 on 35 degrees of freedom
 AIC: 64.896

Number of Fisher Scoring iterations: 18

```
ggplot(muestra_2etapas, aes(x = age, fill = as.factor(responde))) +
  geom_density(alpha = 0.5) +
  labs(title = "Análisis de No Respuesta por Edad",
       subtitle = "Si las curvas son muy diferentes, hay un sesgo de selección",
       fill = "¿Respondió?") +
  theme_minimal()
```

Análisis de No Respuesta por Edad

Si las curvas son muy diferentes, hay un sesgo de selección



Estimador BLUP

Muestras no probabilísticas

```
# 1. Creamos la muestra no probabilística (Sesgada)
# Los jóvenes (age < 30) tienen 5 veces más probabilidad de participar
muestra_no_prob <- bank_data %>%
  mutate(prob_participar = ifelse(age < 30, 0.5, 0.1)) %>%
  slice_sample(n = 500, weight_by = prob_participar)

# 2. Comparamos las medias
media_real <- mean(bank_data$balance)
media_no_prob <- mean(muestra_no_prob$balance)

cat("Media Real Poblacional: ", media_real, "\n")
```

Media Real Poblacional: 1422.658

```
cat("Media Muestra No Probabilística: ", media_no_prob, "\n")
```

Media Muestra No Probabilística: 1310.012

```
cat("Sesgo de Selección: ", media_no_prob - media_real, "\n")
```

Sesgo de Selección: -112.6458

Estimacion en dominios:

```
# Supongamos que nuestro "Dominio" de interés son los clientes con educación 'tertiary'

# 1. Creamos el subconjunto del diseño (mantiene la estructura de pesos y clusters)
diseno_terciario <- subset(diseno_final, education == "tertiary")

# 2. Estimamos la media para ese dominio
res_dominio <- svymean(~balance, diseno_terciario)

# 3. Comparamos con la media global
print(res_dominio)
```

	mean	SE
balance	1790.3	598.5

```
print(svymean(~balance, diseno_final))
```

	mean	SE
balance	1522.5	244.53

```
# Saldo promedio por tipo de trabajo (Dominios)
reporte_trabajos <- svyby(~balance, by = ~job,
                        design = diseno_final,
                        FUN = svymean)

print(reporte_trabajos)
```

	job	balance	se
admin.	admin.	2406.1714	1.414164e+03
blue-collar	blue-collar	1209.3930	3.810012e+02
entrepreneur	entrepreneur	496.6667	2.842171e-14

housemaid	housemaid	443.0000	5.684342e-14
management	management	1282.7095	6.373830e+02
retired	retired	409.6589	2.190566e+02
self-employed	self-employed	2341.9066	1.593456e+03
services	services	1756.9217	8.963064e+02
technician	technician	1827.8722	4.362697e+02
unemployed	unemployed	726.0066	4.382679e+02
unknown	unknown	6836.0000	0.000000e+00

```
# Visualización de Dominios con sus Intervalos de Confianza
ggplot(reporte_trabajos, aes(x = reorder(job, balance), y = balance)) +
  geom_point(size = 3, color = "darkgreen") +
  geom_errorbar(aes(ymin = balance - 1.96*se, ymax = balance + 1.96*se), width = 0.2) +
  coord_flip() +
  labs(title = "Estimación por Dominios: Saldo por Profesión",
       subtitle = "Los intervalos reflejan la incertidumbre de cada subgrupo",
       x = "Profesión", y = "Saldo Promedio") +
  theme_minimal()
```

